

Báo cáo training Data Analyst

Họ và tên	:	Nguyễn Hoàng Yên
Ngày tháng	:	21/05/2026
Người hướng dẫn	:	

Mục lục

1. Cơ sở dữ liệu

1.1. Khái niệm

Cơ sở dữ liệu (CSDL - database) là tập hợp của nhiều dữ liệu được tổ chức quản lý (thêm, xóa, sửa) nhằm mục đích lưu trữ, truy xuất, cập nhật cho nhiều nhu cầu công việc khác nhau. Được dùng để phục vụ nhiều đầu mối khác nhau cùng tác động đến nó.

CSDL thường được tổ chức thành 3 lớp:

- Vật lý: cơ bản nhất, nơi dữ liệu được lưu trữ;
 - Logic: đảm bảo sự nhất quán và tính đúng sai của điều kiện ràng buộc;
 - Giao tiếp: mô hình hóa cấu trúc của dữ liệu theo cách con người có thể hiểu được dễ dàng, mô hình phổ biến của lớp giao tiếp là cấu trúc bảng.
- + Một mẫu dữ liệu được xem là 1 hàng (row/record);
- + Từng phần chi tiết của 1 mẫu dữ liệu được lưu trữ ở từng cột riêng biệt của hàng đó (column).

1.2. Chuẩn hóa cơ sở dữ liệu

Có 2 vấn đề cơ bản dẫn tới tối ưu hóa CSDL:

- Cấu trúc dữ liệu không đảm bảo được tính nhất quán / lưu trữ quá nhiều thông tin thừa, cần chuẩn hóa về tính nhất quán.
- Cấu trúc lưu trữ quá phức tạp để đọc thông tin, cần chuẩn hóa cấu trúc dữ liệu về mặt tốc độ.

Khóa chính (primary key): đảm bảo sự xác định chính xác một hàng dữ liệu mà không nhầm lẫn tới bất kỳ hàng nào khác

- Dữ liệu từng hàng là duy nhất;
- Không bao giờ thay đổi giá trị;
- Ít tốn dung lượng nhất;
- Nếu không có cột nào thỏa mãn 3 câu hỏi trên thì tạo 1 cột khóa chính thỏa mãn cả 3 nguyên tắc, thường là mã, hoặc là sự kết hợp giữa nhiều cột.

Khóa phụ (foreign key): nơi khai báo giá trị của khóa chính.

1.3. Cơ sở dữ liệu quan hệ

Cơ sở dữ liệu quan hệ là CSDL là ở đó nó đảm bảo mối liên kết giữa các khóa phụ và khóa chính luôn được chính xác. Sự đảm bảo này được giữ vững thông qua lớp LOGIC. Mọi thao tác có thể làm sai lệch mối quan hệ sẽ đều bị ngăn chặn. Có 3 loại quan hệ chính:

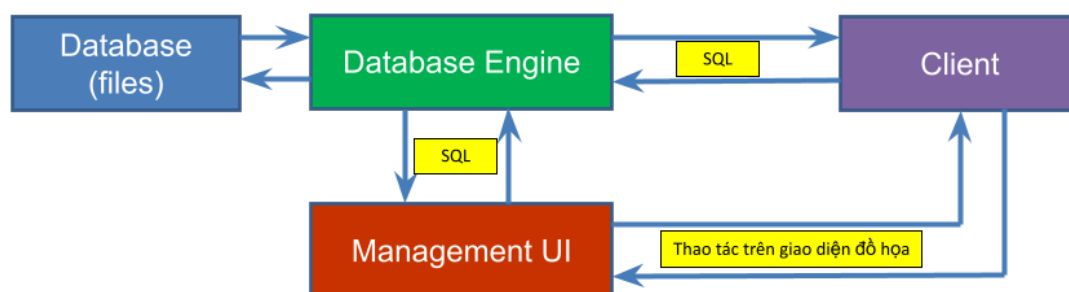
- Quan hệ “Một - Một”, quan hệ “Một - Nhiều”: thông qua sự tham chiếu giữa khóa chính và khóa phụ;
- Quan hệ “Nhiều - nhiều”: thể hiện qua 1 bảng trung gian.

1.4. Hệ quản trị cơ sở dữ liệu quan hệ

Hệ quản trị cơ sở dữ liệu (DBMS) là một phần mềm cho phép tạo lập các CSDL cho các ứng dụng khác nhau và điều khiển mọi truy cập tới các CSDL đó. Nghĩa là, hệ quản trị CSDL cho phép định nghĩa (xác định kiểu, cấu trúc, ràng buộc dữ liệu), tạo lập (lưu trữ dữ liệu trên các thiết bị nhớ) và thao tác (truy vấn, cập nhật, kết xuất, ...) các CSDL cho các ứng dụng khác nhau.

Gồm có:

- + *Database*: là cơ sở dữ liệu, nơi chứa dữ liệu, là lớp vật lý thấp nhất trong 4 lớp cấu tạo của CSDL
- + *Database Engine*: là một phần mềm dịch vụ để quản trị cơ sở dữ liệu, làm cầu nối giao tiếp giữa database với bên ngoài (đọc dữ liệu ra, ghi dữ liệu vào, cập nhật dữ liệu) và chịu trách nhiệm trong việc đảm bảo vận hành lớp logic một cách chính xác.
- + *SQL (Structured Query Language)*: ngôn ngữ riêng dùng để giao tiếp với Database Engine của hệ quản trị CSDL. Database Engine chỉ hiểu và thực hiện các yêu cầu thông qua loại ngôn ngữ này.
- + *Management UI Tools*: giao diện thao tác với dữ liệu mà không cần phải sử dụng ngôn ngữ SQL.



Một số hệ quản trị CSDL quan hệ phổ biến:

- *MySQL*: MySQL là một hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) mã nguồn mở, được sử dụng rộng rãi để lưu trữ, quản lý và truy xuất dữ liệu. MySQL hoàn toàn miễn phí nằm trong nhóm LAMP (Linux – Apache – MySQL – PHP).

- *PostgresQL*: PostgreSQL là một hệ thống quản trị cơ sở dữ liệu quan hệ và đối tượng (object-relational database management system) miễn phí và nguồn mở (RDBMS) tiên tiến nhất hiện nay. Được thiết kế để xử lý một loạt các khối lượng công việc lớn, từ các máy tính cá nhân đến kho dữ liệu hoặc dịch vụ Web có nhiều người dùng đồng thời.
- *Microsoft SQL Server*: có tính phí và thường được sử dụng trên máy chủ cài hệ điều hành Window. Ngôn ngữ truy vấn hỗ trợ rất nhiều tính năng như một ngôn ngữ lập trình và được gọi là Transact - SQL (T-SQL). Phần mềm Management UI có tên là MSSQL Server Management Studio.
- *Oracle Database*: hệ quản trị CSDL quan hệ rất to lớn với nhiều tính năng quản trị dữ liệu và bảo vệ dữ liệu dày đặc. Thường được dùng cho những nhu cầu an ninh và an toàn cao như ngân hàng, tín dụng,...

1.5. Index

Index là hệ thống đánh dấu vị trí những nội dung bên trong một bảng dữ liệu, giúp Database Engine tiếp cận nhanh tới dữ liệu cần đọc mà không cần phải tìm từng dữ liệu. Index chỉ có ý nghĩa với truy vấn đọc dữ liệu có điều kiện.

Các bước thực hiện truy vấn dữ liệu cơ bản của hệ quản trị CSDL:



Các loại Index:

- *Clustered Index*: Là Index chỉ được tạo bởi hệ thống. Một Index chỉ định vị tới chính xác một nội dung duy nhất, các khóa chính (primary key) sẽ được hệ thống mặc định sử dụng làm Clustered Index.
- *Nonclustered Index*: Là Index mở rộng, một phần được tạo bởi hệ thống, một phần được người dùng tạo. Có thể định vị 1 Index tới nhiều hơn 1 Nội dung. Theo mặc định thì khóa phụ (foreign key) sẽ được hệ thống dùng làm Nonclustered Index.

→ Khi truy vấn, luôn phải thêm Index vào.

Tài liệu:

- <https://viblo.asia/p/he-quan-tri-co-so-du-lieu-bJzKmzMkZ9N>
- <https://topdev.vn/blog/gioi-thieu-ve-mysql/>

- https://youtube.com/playlist?list=PL2_xbcFZM80PHczxl-jlOgKA4yus3kWwu&si=WEwsdJjP0jGNbWfh
- <https://viblo.asia/p/tim-hieu-he-quan-tri-co-so-du-lieu-postgresql-m68Z0eLdlkG>

2. Các ngôn ngữ lập trình

2.1. SQL

a. Khái niệm & dữ liệu:

SQL (Structured Query Language) là ngôn ngữ truy vấn có cấu trúc. Được sử dụng để truy xuất dữ liệu hoặc giao diện khác như tạo, xóa trong cơ sở dữ liệu, lấy các hàng và sửa đổi các hàng .v.v. với cơ sở dữ liệu quan hệ.

Một số loại dữ liệu khác nhau có sẵn trong PostgreSQL bao gồm: Text, Numeric, Temporal, Boolean. Có 5 kiểu dữ liệu tạm thời chính: **DATE**, **TIME**, **TIMESTAMP**, **TIMESTAMPTZ**, **INTERVAL**.

b. Khởi tạo bảng

- Lệnh **INSERT**:

INSERT INTO bảng(cột1, cột2, ...) **VALUES** (giá_trị1, giá_trị2, ...);

```
1  --Insert information of employees to database
2  INSERT INTO employees(emp_id,emp_name,hire_date,salary,com)
3  VALUES ('SE20001','Harry','2017-01-01',2000,0.03);
4  INSERT INTO employees(emp_id,emp_name,hire_date,salary,com)
5  VALUES ('SE20002','Hermione','2018-01-01', 2000.55, 0.04);
6  INSERT INTO employees(emp_id,emp_name,hire_date,salary,com)
7  VALUES ('SE20003','Ron','2017-02-01', 3000, 0.03);
8  INSERT INTO employees(emp_id,emp_name,hire_date,salary,com)
9  VALUES ('SE20004','Gimmy','2019-01-03',3000,0.03);
10
11 --Select all the information in the table employees
12 SELECT * FROM employees;
```

- Lệnh **ALTER TABLE**

- + **ADD COLUMN**:

ALTER TABLE tên_bảng

ADD COLUMN tên_cột_mới_1 kiểu_dữ_liệu ràng_buộc **DEFAULT** giá_trị,

ADD COLUMN tên_cột_mới_2 kiểu_dữ_liệu ràng_buộc **DEFAULT** giá_trị,

...;

```

1  --Implement the missing code, denoted by ellipses. You may not modify the pre-existing code.
2
3  --Column description should be set to "TBD" (for To Be Defined).
4  ALTER TABLE restaurants
5  ADD COLUMN description TEXT DEFAULT 'TBD';
6  --Column active should be set to 1.
7  ALTER TABLE restaurants
8  ADD COLUMN active SMALLINT DEFAULT 1;
9
10 -- Show the information of all restaurants ordering by id.
11 SELECT * FROM restaurants ORDER BY id;

```

+ Thay đổi kiểu dữ liệu:

```

ALTER TABLE tên_bảng
ALTER COLUMN tên_cột_1 [SET DATA] TYPE kiểu_dữ_liệu_mới,
ALTER COLUMN tên_cột_2 [SET DATA] TYPE kiểu_dữ_liệu_mới,
...;

```

```

1  /* Implement the missing code, denoted by ellipses. You may not modify the pre-existing code. */
2
3  ALTER TABLE employees
4  ALTER COLUMN emp_id TYPE VARCHAR(7);
5
6  --Insert information of employees to database
7  INSERT INTO employees(emp_id,emp_name,hire_date,salary,com)
8  VALUES
9  ('SE00001','Marie','2017-01-01',2000,0.03),
10 ('SE00002','Harry','2017-01-01',2000.55,0.03),
11 ('SE00003','Maries','2017-01-01',3000,0.03),
12 ('SE00004','Ron','2017-01-01',3000,0.03),
13 ('003','Hermione','2017-01-01',3000,0.03);
14
15 --Show all the information of all employees ascendingly
16 SELECT * FROM employees ORDER BY emp_id ASC;
17

```

+ **RENAME COLUMN**

```
ALTER TABLE tên_bảng RENAME COLUMN tên_cột TO tên_cột_mới;
```

c. Những ràng buộc

- **Lệnh ràng buộc: ADD CONSTRAINT**

Ý tưởng của cơ sở dữ liệu là sắp xếp dữ liệu theo một cấu trúc nhất định - một mô hình được xác định trước, nơi sử dụng các kiểu dữ liệu, các mối quan hệ và các quy tắc lên cấu trúc đấy. Nhìn chung, những quy tắc này được gọi là các ràng buộc toàn vẹn (integrity constraints). Có thể được chia thành ba loại:

1. Ràng buộc thuộc tính, ví dụ kiểu dữ liệu trên cột.
2. Các ràng buộc khóa, ví dụ các khóa chính.
3. Các ràng buộc tham chiếu, được thi hành thông qua các khóa ngoại.

```

ALTER TABLE table_name
ADD CONSTRAINT constraint_name constraint_definition;

```

****Ràng buộc thuộc tính đặc biệt***

Có 2 thuộc tính đặc biệt: các ràng buộc **NOT NULL** và **UNIQUE**.

- + Giá trị **NULL** nghĩa là một giá trị không được xác định hoặc hoàn toàn không tồn tại.

```
ALTER TABLE tên_bảng ALTER COLUMN tên_cột SET NOT NULL;
```

- + Ràng buộc **UNIQUE** trên một cột đảm bảo rằng không có sự trùng lặp trong cột đấy.

```
ALTER TABLE tên_bảng ADD CONSTRAINT tên_ràng_buộc  
UNIQUE(tên_cột);
```

- Lệnh thêm khóa chính/ngoại
- + Khóa chính

```
CREATE TABLE TABLE (  
    cột_1 kiểu_dữ_liệu PRIMARY KEY,  
    cột_2 kiểu_dữ_liệu,  
    ...  
);
```

- + Khóa ngoại

--Cách 1

```
CREATE TABLE bảng_con(  
    c1 kiểu_dữ_liệu PRIMARY KEY,  
    c2 kiểu_dữ_liệu REFERENCES bảng_bố(p2)  
);
```

--Cách 2

```
CREATE TABLE bảng_con(  
    c1 kiểu_dữ_liệu PRIMARY KEY,  
    c2 kiểu_dữ_liệu ,  
    FOREIGN KEY (c2) REFERENCES bảng_bố(p2)  
);
```

- Toàn vẹn tham chiếu

```
CREATE TABLE a  
    id INTEGER PRIMARY KEY,  
    column_a VARCHAR(50),  
    ...,  
    b_id INTEGER REFERENCES b (id) ON DELETE NO ACTION/CASCADE  
);
```

- Lệnh **DROP CONSTRAINT**

```
ALTER TABLE tên_bảng DROP CONSTRAINT tên_ràng_buộc;
```

d. Sửa đổi dữ liệu

- Lệnh **UPDATE**

```
UPDATE bảng
```

```
SET cột_1 = giá_trị1, cột_2 = giá_trị2 ,...
```

```
WHERE điều_ kiện;
```

```
1  -- Set professor_id to professors.id where first_name, last_name correspond to rows in professors
2  UPDATE affiliations
3  SET
4  professor_id = professors.id
5  FROM professors
6  WHERE
7  affiliations.first_name = professors.first_name
8  AND affiliations.last_name = professors.last_name;
9  -- Have a look at the affiliations table
10 SELECT * FROM affiliations;
```

- Các lệnh xóa

- + **DROP COLUMN**

```
ALTER TABLE tên_bảng DROP COLUMN tên_cột;
```

```
ALTER TABLE tên_bảng DROP COLUMN tên_cột CASCADE; (xóa cột và tất cả các  
đối tượng liên quan)
```

```
ALTER TABLE tên_bảng DROP COLUMN IF EXISTS tên_cột; (check xem cột  
có tồn tại không)
```

- + **DROP TABLE**

```
DROP TABLE [IF EXISTS] tên_bảng [CASCADE | RESTRICT];
```

Để tránh lỗi này, có thể sử dụng tham số **IF EXISTS**.

- + **DELETE** (để xóa dữ liệu khỏi bảng)

```
DELETE FROM bảng WHERE điều_ kiện;
```

- + **TRUNCATE** (xóa tất cả các hàng khỏi một tập hợp các bảng)

```
TRUNCATE TABLE tên_bảng;
```

e. Chuẩn hóa dữ liệu

- Dạng chuẩn đầu tiên - **1st Normal Form (1NF)**: Các giá trị bảng phải là nguyên tử, điều này có nghĩa là giá trị không thể được chia thành các đơn vị nhỏ hơn.
- Dạng chuẩn 2 - **2NF**: Một cơ sở dữ liệu ở **2NF** khi nó đạt được tiêu chuẩn **1NF** và các cột không khóa chỉ phụ thuộc vào **Khóa chính** của bảng.
- Dạng chuẩn 3 - **3NF**:
- Đầu tiên, đáp ứng các yêu cầu của **2NF**.

- Không có sự tồn tại của phụ thuộc bắc cầu trong bảng. Điều này có nghĩa là các cột không khóa trong bảng chỉ phụ thuộc vào khóa chính.

f. Truy vấn dữ liệu

- **SELECT:**

Một trong những tác vụ phổ biến nhất khi làm việc với PostgreSQL là truy vấn dữ liệu từ các bảng bằng cách sử dụng câu lệnh **SELECT**.

```
SELECT cột_1, cột_2, ... FROM tên_bảng;
```

- Tên thay thế cho bảng:

```
SELECT danh_sách_cột FROM tên_bảng AS tên_thay_thế;
```

- Cast Operator (dùng để chuyển đổi một giá trị của một loại dữ liệu thành một loại dữ liệu khác)

```
CAST( biểu_thức AS loại_dữ_liệu_đích );
```

hoặc

```
biểu_thức::loại
```

- Hàm **ROUND()** (làm tròn một giá trị số thành số nguyên gần nhất hoặc một số có số vị trí thập phân)

```
ROUND (source [ , n ] )
```

- Mệnh đề **ORDER BY** (sắp xếp các hàng được trả về từ câu lệnh **SELECT** theo thứ tự tăng dần hoặc giảm dần dựa trên các tiêu chí đã chỉ định)

```
SELECT cột_1, cột_2  
FROM tên_bảng  
ORDER BY cột_1 ASC, cột_2 DESC;
```

- Mệnh đề **SELECT DISTINCT** (xóa các hàng trùng lặp khỏi tập kết quả)

```
SELECT DISTINCT cột_1, cột_2 FROM tên_bảng;
```

g. Lọc dữ liệu

- Mệnh đề **WHERE**

```
SELECT danh_sách_chọn  
FROM tên_bảng  
WHERE điều_ kiện;
```

Mệnh đề **WHERE** xuất hiện ngay sau mệnh đề **FROM** của câu lệnh **SELECT**. Mệnh đề **WHERE** sử dụng điều kiện để lọc các hàng được trả về từ câu lệnh **SELECT**. Điều kiện

phải đánh giá là đúng, sai hoặc không biết. Nó có thể là biểu thức Boolean hoặc kết hợp các biểu thức Boolean bằng toán tử **AND** và **OR**.

Nếu muốn khớp một chuỗi với bất kỳ chuỗi nào trong một danh sách, sử dụng toán tử **IN**.

```
SELECT tên_cột1, tên_cột2
FROM tên_bảng
WHERE tên_cột1 IN ('...', '...');
```

- Toán tử **LIKE** / **NOT LIKE** (tìm các kết quả khớp/khác với mẫu). PostgreSQL cung cấp hai ký tự đại diện như sau:
 - Dấu phần trăm (%) để khớp với bất kỳ chuỗi ký tự nào.
 - Dấu gạch dưới (_) để khớp với bất kỳ ký tự đơn nào.

```
1  /* Implement the missing code, denoted by ellipses.
2  You may not modify the pre-existing code. */
3
4  SELECT
5  id, name, surname
6  FROM
7  suspect
8  WHERE
9  height <= 170
10 AND (name LIKE 'B%' OR name LIKE 'b%')
11 AND (surname LIKE 'Gre_n' OR surname LIKE 'gre_n')
12 ORDER BY id ASC;
```

PostgreSQL cung cấp toán tử **ILIKE** hoạt động như toán tử **LIKE**. Ngoài ra, toán tử **ILIKE** khớp với giá trị không phân biệt chữ hoa chữ thường.

- Toán tử **BETWEEN** (khớp một giá trị với một phạm vi các giá trị)

```
giá_trị BETWEEN thấp AND cao;
giá_trị NOT BETWEEN thấp AND cao;
```

- Mệnh đề **LIMIT** (câu lệnh trả về **n** hàng được tạo bởi truy vấn)

```
SELECT * FROM tên_bảng LIMIT n;
```

→ Luôn sử dụng mệnh đề **ORDER BY** để kiểm soát thứ tự hàng. Trong trường hợp muốn bỏ qua một số hàng trước khi trả về **n** hàng, cần sử dụng mệnh đề **OFFSET** được đặt sau mệnh đề **LIMIT**:

```
SELECT * FROM tên_bảng
LIMIT n OFFSET m;
```

- Toán tử **IS NULL** (thiếu hoặc không có thông tin)

giá_trị IS NULL / IS NOT NULL

h. Gộp nhiều bảng

- Mệnh đề **INNER JOIN**

```
SELECT A.pka, A.c1, B.pkb, B.c2
FROM A INNER JOIN B ON A.pka = B.fka;
```

→ Mệnh đề **INNER JOIN** trả về các hàng trong bảng A có các hàng tương ứng trong bảng B.

- Hàm **COALESCE** (đánh giá các đối số từ trái sang phải và trả về đối số đầu tiên không phải là **NULL**)

```
COALESCE (đối_số1, đối_số2, ...);
```

- Mệnh đề **LEFT JOIN**

```
SELECT A.pka, A.c1, B.pkb, B.c2
FROM A LEFT JOIN B ON A.pka = B.fka;
```

→ Mệnh đề **LEFT JOIN** trả về tất cả các hàng trong bảng bên trái (A) được kết hợp với các hàng trong bảng bên phải (B) mặc dù có thể tồn tại trường hợp là không có hàng tương ứng trong bảng bên phải (B)

- Self-join (truy vấn trong đó một bảng tự join với chính nó)

Để hình thành việc self-join, cần chỉ định cùng một bảng hai lần với các bí danh (alias) khác nhau, thiết lập so sánh và loại bỏ các trường hợp trong đó một giá trị sẽ bằng chính nó.

```
SELECT danh_sách_cột FROM A a1
INNER JOIN A b1 ON điều_kiện_join;
```

```
1  /* Implement the missing code, denoted by ellipses.
2  You may not modify the pre-existing code.*/
3
4  --Select appropriate fields with aliases
5  SELECT
6  c1.country_code,
7  c1.size AS size2010,
8  c2.size AS size2015
9  --From populations JOIN to itself, match on country code and year
10 FROM
11 populations c1
12 INNER JOIN populations c2
13 ON c1.country_code = c2.country_code
14 WHERE c1.year = '2010'
15 |   AND c2.year = '2015';
```


i. Phân nhóm dữ liệu

- Hàm **SUM** (trả về tổng giá trị hoặc giá trị riêng biệt)

```
SUM(DISTINCT cột)
```

- Hàm **MIN/MAX/AVG**

```
SELECT MIN(cột) FROM bảng;
```

```
SELECT MAX(cột) FROM bảng;
```

```
SELECT AVG(cột) FROM bảng;
```

- Mệnh đề **GROUP BY**

```
SELECT cột_1, hàm_tổng_hợp(cột_2)
```

```
FROM tên_bảng
```

```
GROUP BY cột_1;
```

- Mệnh đề **HAVING**

```
SELECT cột_1, hàm_tổng_hợp(cột_2)
```

```
FROM tên_bảng
```

```
GROUP BY cột_1
```

```
HAVING điều_ kiện;
```

→ Mệnh đề **HAVING** đặt điều kiện cho các hàng nhóm được tạo bởi mệnh đề **GROUP BY** sau khi mệnh đề **GROUP BY** áp dụng

j. Truy vấn con (Subquery Technique)

Truy vấn con là một truy vấn được lồng bên trong một truy vấn khác. Truy vấn bên trong ngoặc được gọi là truy vấn con hoặc truy vấn bên trong. Truy vấn có chứa truy vấn con được gọi là truy vấn bên ngoài.

```
1  /*Implement the missing code, denoted by ellipses.
2  You may not modify the pre-existing code. */
3  SELECT first_name, last_name
4  FROM actor
5  WHERE actor_id IN (
6      SELECT actor_id
7      FROM film_actor
8      WHERE film_id = (
9          SELECT film_id
10         FROM film
11         WHERE title = 'ACADEMY DINOSAUR'
12     )
13 )
14 ORDER BY first_name ASC, last_name ASC;
```

- Toán tử **EXISTS** (kiểm tra sự tồn tại của các hàng trong truy vấn con)

```
SELECT
    column_1
FROM
    table_1
```

```
WHERE  
EXISTS ( SELECT  
          1  
        FROM  
          table_2  
        WHERE  
          column_2 = table_1.column_1);
```

→ Nếu truy vấn con trả về **NULL**, kết quả của **EXISTS** là false.

NOT EXISTS ngược lại với **EXISTS**, nghĩa là nếu truy vấn con không trả về hàng, thì **NOT EXISTS** trả về true. Nếu truy vấn con trả về bất kỳ hàng nào, **NOT EXISTS** trả về false.

Tài liệu:

- <https://viblo.asia/p/tim-hieu-ve-sql-ngon-ngu-truy-van-co-cau-truc-bJzKmwr6l9N>
- <https://codelearn.io/learning/lam-quen-voi-sql>



2.2. Python

a. Khái niệm

Python là một ngôn ngữ lập trình bậc cao, mã nguồn mở và đa nền tảng. Python được sử dụng rộng rãi để phát triển các ứng dụng web, phát triển phần mềm, khoa học dữ liệu và máy học (ML).

- **Biến:** Được sử dụng để lưu trữ dữ liệu trong chương trình, được tạo ra khi bắt đầu gán giá trị cho nó. Dữ liệu được lưu trữ trong một biến có thể thay đổi trong khi chạy chương trình.

```
# Khai báo biến a và gán giá trị cho a = 5
a = 5
# Khai báo biến b và gán giá trị cho b = 7
b = 7
print("a + b =", a + b)
```

- **Kiểu dữ liệu:** Một số kiểu dữ liệu cơ bản trong Python:
 - **int:** lưu trữ các số nguyên.
 - **float:** lưu trữ các biến kiểu số thực.
 - **bool:** lưu trữ các giá trị luận lý (**True** hoặc **False**)
 - **str:** lưu trữ các chuỗi ký tự ("**Viet Nam**", "**Codelearn**", ...)

Một số lưu ý khi đặt tên biến:

- Tên biến *có thể* được bắt đầu bằng ký tự `_` hoặc bất cứ chữ cái nào (in hoa hoặc in thường).
- Tên biến **không** được bắt đầu bằng chữ số. Ví dụ tên biến `100invalid` là không hợp lệ.
- Tên biến **không** được bao gồm dấu cách trong đó.
- **Không** sử dụng các từ khóa có sẵn trong Python như `print`, `if`, `else`... để đặt tên cho biến. Ví dụ tên biến `print` là không hợp lệ, tên biến `print1`, `print_2`, `printHello`... là hợp lệ.

b. Các hàm/câu lệnh cơ bản

- Hàm `print()` (để thể hiện ra màn hình thông tin nào đó)

```
print(thông_tin)
```

- Comment/chú thích:

- + Comment trên 1 dòng: được bắt đầu với ký tự `#`
- + Comment trên nhiều dòng: bắt đầu bằng 3 dấu nháy đơn (`'''`) hoặc 3 dấu nháy kép (`"""`) và kết thúc cũng bằng 3 dấu nháy đơn (`'''`) hoặc 3 dấu nháy kép (`"""`)
- Hàm `input()`

```
# Nhập dữ liệu cho biến name từ bàn phím
name = input()
print("Hello " + name)
```

Trong Python, dữ liệu nhập từ bàn phím luôn có kiểu là `str` (hàm `input()` trả về kiểu `str`), mà biến kiểu `str` thì không thể tính toán được. Do đó bạn cần chuyển kiểu dữ liệu của biến `age` về kiểu `int`, để làm việc này bạn cần sử dụng hàm `int()`.

- Toán tử cơ bản
- Toán tử số học

```
print('x + y =', x+y)
print('x - y =', x-y)
print('x * y =', x*y)
print('x / y =', x/y)
print('x % y =', x % y)
print('x // y =', x//y)
print('x ** y =', x**y)
```

- Toán tử so sánh (kết quả là `True` hoặc `False`: `>`, `<`, `==`, `!=`, `>=`, `<=`)
- Toán tử gán trong Python (`=`, `+=`, `-=`, `*=`, `/=`,...)

```
1 a = int(input())
2 Total = int(input())
3 Total += a # Using += Operator
4 print("The Value of the Total after using += Operator is:", Total)
5 Total -= a # Using -= Operator
6 print("The Value of the Total after using -= Operator is:", Total)
7 Total *= a # Using *= Operator
8 print("The Value of the Total after using *= Operator is:", Total)
9 Total /= a # Using /= Operator
10 print("The Value of the Total after using /= Operator is:", Total)
11 Total **= a # Using **= Operator
12 print("The Value of the Total after using **= Operator is:", Total)
13 Total %= a # Using %= Operator
14 print("The Value of the Total after using %= Operator is:", Total)
15 Total /= a # Using /= Operator
16 print("The Value of the Total after using /= Operator is:", Total)
```

- Toán tử membership (kiểm xem một giá trị có nằm trong giá trị khác hay không: `in/ not in`)
- Toán tử định danh (`identity`) (xác định xem hai biến có đang trỏ tới cùng một đối tượng hay không: `is/ is not`)
- Toán tử logical (logic): `not, or, and`
- Mệnh đề `if-else` được dùng để kiểm tra một điều kiện có đúng hay không, nếu đúng thì khối lệnh bên trong `if` sẽ được thực thi, ngược lại nếu điều kiện là sai thì khối lệnh bên trong `else` sẽ được thực thi:

```
if condition:
    # Nếu condition đúng thì khối lệnh này sẽ được thực thi
else:
    # Nếu condition sai thì khối lệnh này sẽ được thực thi

- Mệnh đề elif
a = int(input())
b = int(input())
if a == b:
    print("a = b")
elif a > b:
    print("a > b")
else:
    print("a < b")
```

- Vòng lặp:
- + `while`: lặp lại một hành động cho tới khi điều kiện lặp không còn thỏa mãn nữa.

```
while condition:
    # Khối lệnh này sẽ được thực thi nếu condition còn đúng

+ for: lặp qua một tập hợp cho trước, thường được sử dụng với hàm range().
for i in range(a, b):
    print(i)
```

- + Câu lệnh điều khiển vòng lặp

`break`: khi chương trình gặp câu lệnh `break` thì vòng lặp sẽ không được thực thi tiếp.

`continue`: các đoạn code bên dưới câu lệnh này trong vòng lặp sẽ không được thực thi.

- Hàm `round()`: làm tròn các số thực.

```
round(number, ndigits)
```

- `list`: kiểu dữ liệu cho phép lưu trữ nhiều kiểu dữ liệu khác.

```
# Tạo ra list để lưu trữ các số nguyên
```

```
list1 = [1, 2, 3]
```

```
# Tạo ra list để lưu trữ các chuỗi ký tự
```

```
list2 = ["Viet", "Tuan", "Duong"]
```

```
# Bạn cũng có thể tạo ra một list lưu trữ các kiểu dữ liệu khác nhau
```

```
list3 = [7, 3.5, "Codelearn"]
```

- Hàm `append()`: để thêm một phần tử vào cuối của list.

```
1 n = int(input())
2 lst = []
3 for n in range(n):
4     lst.append(int(input()))
5
6 sum = 0
7 for i in lst:
8     sum += i
9 print(sum)
```

Các hàm thường được sử dụng trong `list`: `len()`, `reverse()`, `pop()`, ...

- Chuỗi: Hàm `len()`, phương thức `lower/upper()`, `isalnum/isalpha/isnumeric()`, `split()`, `join()`, `replace()`, ...
- Cắt chuỗi trong Python (Slice a string in Python): để lấy ra một dãy các ký tự liên tiếp trong chuỗi

```
s = 'Python String'
```

```
print(s[0:2])
```

```
print(s[3:5])
```

```
print(s[7:])
```

```
print(s[:6])
```

```
print(s[7:-4])
```

- Tạo và sử dụng hàm: khai báo một hàm dùng từ khóa `def`

```
def <tên_hàm>:
```

```
    # Các đoạn code trong thân hàm
```

- + Truyền các tham số vào hàm và trả về kết quả bằng từ khóa `return` `<giá_trị_trả_về>`.

```
def sum_of_list(lst):  
    answer = 0  
    for v in lst:  
        answer += v  
    return answer
```

```
print(sum_of_list([3, 4, 2]))  
print(sum_of_list([8, 4, 7]))  
print(sum_of_list([1, 2, 3]))
```

Tài liệu:

- <https://topdev.vn/blog/python-la-gi/>
- <https://codelearn.io/learning/python-co-ban>

